

```
--- djangoapi\scripts\crop\DjangoModels\lineas_riego\lineas_riegoDJ.py ---
```

```
from crop.models import LineasRiego
from scripts.crop.DjangoModels.tablasDJ import TablasDJ
from django.contrib.gis.geos import GEOSGeometry

class LineasRiegoDJ(TablasDJ):
    def __init__(self):
        super().__init__(LineasRiego)

    def insert(self, d: dict):
        """Inserta una nueva línea de riego con validación espacial."""
        # 1. Snapping y GEOS
        wkb = self._get_snapped_wkb(d['geom'])
        g = GEOSGeometry(wkb, srid=self.srid)

        if not g.valid:
            return {'ok': False, 'message': 'Geometría de línea inválida'}

        # 2. Validación de intersección de interiores (T*****)
        if self._check_interior_intersection(wkb):
            return {'ok': False, 'message': 'La línea solapa el interior de una existente'}

        # 3. Mapeo de campos y cálculo de longitud
        d['geom'] = g
        d['longitud_m'] = g.length # Cálculo automático para LineString

        obj = self.model(**d)
        obj.save()
        return {'ok': True, 'id': obj.id}

    def update(self, d: dict):
        """Actualiza una línea existente excluyendo su propio ID de la validación."""
        try:
            # 1. Recuperar objeto
            obj = self.model.objects.get(id=d['id'])

            # 2. Snapping y validación de nueva geometría
            wkb = self._get_snapped_wkb(d['geom'])
            g = GEOSGeometry(wkb, srid=self.srid)

            if not g.valid:
                return {'ok': False, 'message': 'Geometría inválida'}

            # 3. Validación T***** excluyendo la propia línea
            if self._check_interior_intersection(wkb, exclude_id=d['id']):
                return {'ok': False, 'message': 'La actualización genera una colisión con otra línea'}

            # 4. Actualización de atributos
            obj.material = d.get('material', obj.material)
            obj.estado = d.get('estado', obj.estado)
```

```
obj.diametro_pulg = d.get('diametro_pulg', obj.diametro_pulg)
obj.geom = g
obj.longitud_m = g.length

obj.save()
return {'ok': True, 'message': f'Línea {d["id"]} actualizada correctamente'}

except self.model.DoesNotExist:
    return {'ok': False, 'message': 'ID de línea no encontrado'}
```